Visualization of Network Data

Xu Shirong

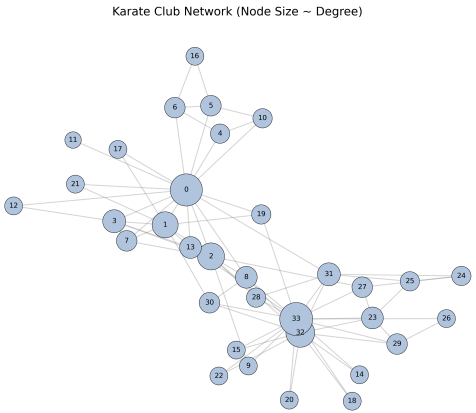
School of Economics, Xiamen University

2026

Outline

1. Part I: Zachary's Karate Club: From dataset introduction to degree distribution and community detection.
2. Part II: Les Miserables Character Network

Dataset Introduction



Basic Facts

Item	Value
Nodes	34
Edges	78
Average degree	4.59
Graph type	Unweighted
Ground truth	Yes
True communities	2

- Nodes are club members.
- Edges are friendship ties.
- A real conflict split the club into two factions.

Part I Workflow

1. Understand the dataset.
2. Draw a simple network visualization.
3. Analyze the node degree distribution.
4. Compare community detection results with the true labels.

Code: Load the Karate Club Dataset

```
G = nx.karate_club_graph()
karate_pos = nx.spring_layout(G, seed=42)
karate_degree = dict(G.degree())

karate_degree_values = list(karate_degree.values()) # list of degrees
karate_degree_min = min(karate_degree_values) # Minimal Degree = 1
karate_degree_max = max(karate_degree_values) # Maximal Degree = 17
```

- We start directly from the built-in NetworkX dataset.
- The spring layout is fixed with a seed so the graph stays reproducible.

Code: Prepare Node Size from Degree

```
karate_node_sizes = {}
for node, degree_value in karate_degree.items():
    karate_node_sizes[node] = (
        450
        + (degree_value - karate_degree_min) * (1800 - 450)
        / (karate_degree_max - karate_degree_min)
    )
# karate_node_sizes is the node size list for visualization
# The above codes rescale node degree to node size.
```

- Node size is computed directly inside the Karate Club section.
- Larger nodes correspond to higher degree.

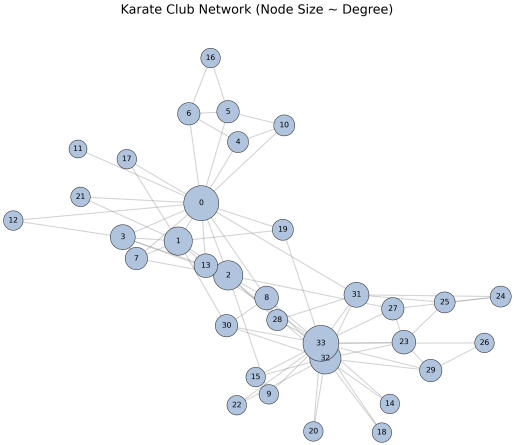
Simple Visualization

- The first graph is a basic structural view of the network.
- Node size already reflects degree.
- At this stage, we do not color by community yet.
- This helps students first focus on overall topology and important nodes.

Code: Draw a Simple Karate Club Visualization

```
plt.figure(figsize=(10, 8))
nx.draw_networkx_edges(G, karate_pos, alpha=0.35,
                       edge_color="gray", width=1.2) # draw the graph edges
nx.draw_networkx_nodes(
    G, karate_pos,
    node_size=[karate_node_sizes[node] for node in G.nodes()],
    node_color="lightsteelblue",
    edgecolors="black", linewidths=0.5
) # draw the graph nodes
nx.draw_networkx_labels(G, karate_pos, font_size=9)
```

Karate Club: Simple Visualization



- What other information can we put on this graph?

Degree Distribution

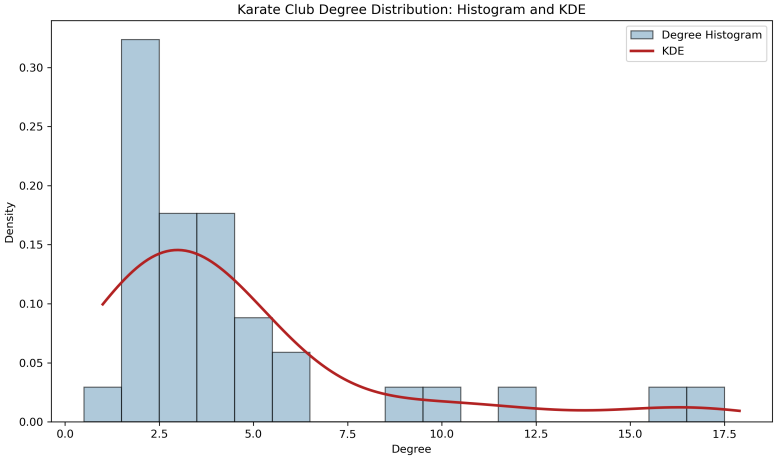
- Degree tells us how many direct neighbors each node has.
- **Question**: What is the degree distribution?
- A histogram summarizes how degree is distributed across the network.
- A KDE curve gives a smoother estimate of the same pattern.
- This is a good step before community detection because it shows local connectivity first.

Code: Karate Degree Histogram and KDE

```
plt.hist(
    karate_degree_values,
    bins=range(min(karate_degree_values),
               max(karate_degree_values) + 2),
    density=True,
    alpha=0.6,
    color="#7aa6c2",
    edgecolor="black",
    align="left"
)

kde = gaussian_kde(karate_degree_values)
plt.plot(x_grid, y_grid, color="#b22222", linewidth=2.4)
```

Karate Club: Degree Histogram and KDE



- When detecting communities, we should investigate what the distribution of degree is under the SBM.

SBM Perspective on Degree Distribution

- Suppose the network were generated by a two-block stochastic block model (SBM).
- Then the degree of a node in one block is the sum of:
- So the degree distribution is not just one simple binomial shape.
- It is a mixture of block-specific random degree distributions.

Stochastic Block Model (SBM)

Model Setup

- Consider a graph with $n = 10000$ nodes.
- Nodes are partitioned into 2 communities:

$$C_1,\ C_2, \quad |C_1| \approx |C_2| \approx 5000$$

- Edge probabilities follow:

$$P = \begin{pmatrix} 0.8 & 0.1 \\ 0.1 & 0.3 \end{pmatrix}$$

Interpretation

- Community 1: dense (strong internal connectivity)
- Community 2: relatively sparse
- Cross-community edges are rare

Degree Distribution: Theoretical Insight

For a node i in community k:

$$\deg(i) = \sum_{j \in C_1} A_{ij} + \sum_{j \in C_2} A_{ij}$$

Approximation:

- If $i \in C_1$:

$$\deg(i) \approx \text{Bin}(5000, 0.8) + \text{Bin}(5000, 0.1)$$
$$\mathbb{E}[\deg(i)] \approx 4500$$

- If $i \in C_2$:

$$\deg(i) \approx \text{Bin}(5000, 0.3) + \text{Bin}(5000, 0.1)$$
$$\mathbb{E}[\deg(i)] \approx 2000$$

Key Insight:

- Degree distribution is a **mixture of two distributions**
- Expect **bimodality**

Simulation Setup

Procedure

- Generate SBM graph with:
  - $n = 10000$
  - 2 communities (equal size)
  - probability matrix:

$$\begin{pmatrix} 0.8 & 0.1 \\ 0.1 & 0.3 \end{pmatrix}$$

- Compute degree of each node:

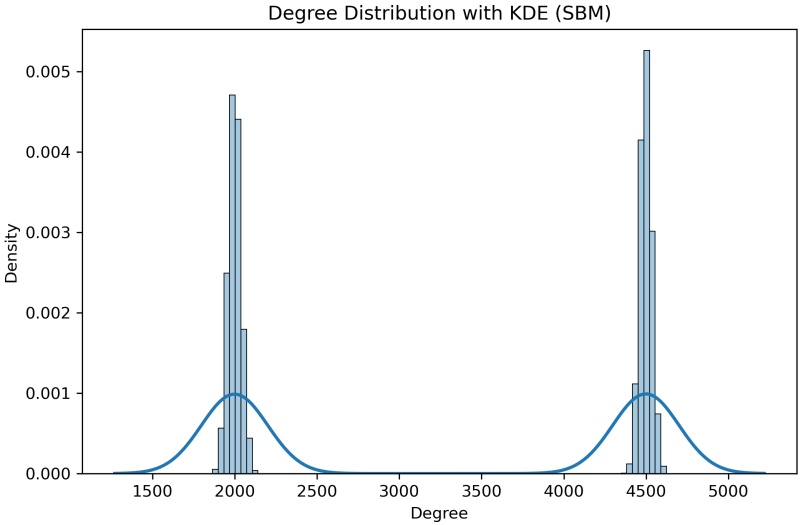
$$\deg(i) = \sum_j A_{ij}$$

- Visualize:
  - Histogram (global)
  - Kernel Density Estimation (KDE)

Goal

- Empirically verify mixture structure

Results: Histogram + KDE



How to Read the SBM Simulation

- The left panel shows one sample graph generated from the fitted two-block SBM.
- The observed Karate Club KDE is overlaid for comparison.
- The dashed vertical lines are the expected degree levels for the two blocks.
- This connects the empirical network to a generative probabilistic model.

Code: Extract the Ground Truth Labels

- This dataset is special because the true split is known.
- The attribute `club` tells us which faction each member belongs to.
- That means we can compare algorithmic community detection with reality.

```
karate_ground_truth = {} # this dataset contains ground-truth labels
for node, data in G.nodes(data=True):
    if data["club"] == "Mr. Hi": # Mr. Hi is the label of one group
        karate_ground_truth[node] = 0
    else:
        karate_ground_truth[node] = 1
```

Code: Compare the Community Detection Result with Ground Truth

```
A = nx.to_numpy_array(G)
features = A
kmeans_raw = KMeans(n_clusters=2, random_state=42, n_init=10)
labels_raw = kmeans_raw.fit_predict(features)
karate_partition_raw = {
    node: labels_raw[i] for i, node in enumerate(G.nodes())
}
```

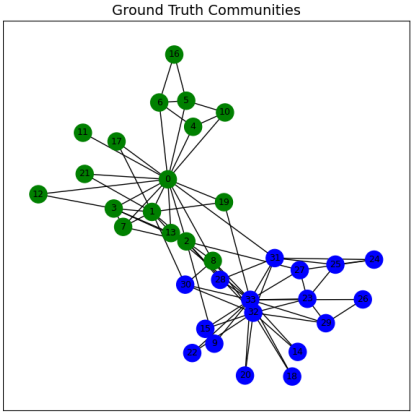
- In this method, we simply treat each i-th row as the feature of i-th node and apply k-means algorithm.
- **Question**: What is the main problem of this method?

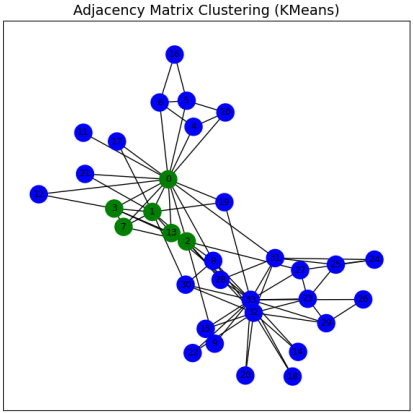
Clustering Result

```
color = ["green","blue"] # use different color for different ndoes
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
nx.draw_networkx_edges(G, karate_pos, ax=axes[0])
nx.draw_networkx_nodes(G, karate_pos, ax=axes[0],
    node_color=[color[karate_ground_truth[node]] for node in G.nodes()])
nx.draw_networkx_labels(G, karate_pos, ax=axes[0], font_size=9)
# ---- Adjacency Clustering ----
nx.draw_networkx_edges(G, karate_pos, ax=axes[1])
nx.draw_networkx_nodes(G, karate_pos, ax=axes[1],
    node_color=[color[1-karate_partition_raw[node]] for node in G.nodes()])
nx.draw_networkx_labels(G, karate_pos, ax=axes[1], font_size=9)
axes[0].set_title("Ground Truth Communities", fontsize=14)
axes[1].set_title("Adjacency Matrix Clustering (KMeans)", fontsize=14)
```

- The clustering accuracy is not good, roughly 67%.

Result

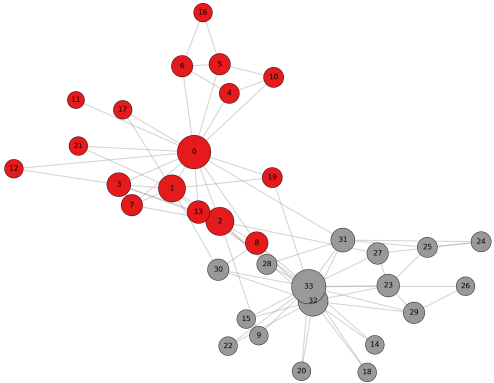

Ground Truth Communities


Adjacency Matrix Clustering (KMeans)

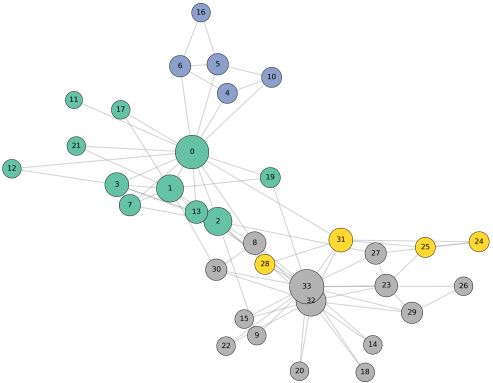
Code: Louvain Community Detection for Karate Club

```
karate_louvain_partition = community_louvain.best_partition(G, random_state=42)
karate_louvain_modularity=community_louvain.modularity(karate_louvain_partition,G)
karate_louvain_community_count = len(set(karate_louvain_partition.values()))
```

Karate Club Ground Truth Communities



Karate Club Louvain Communities



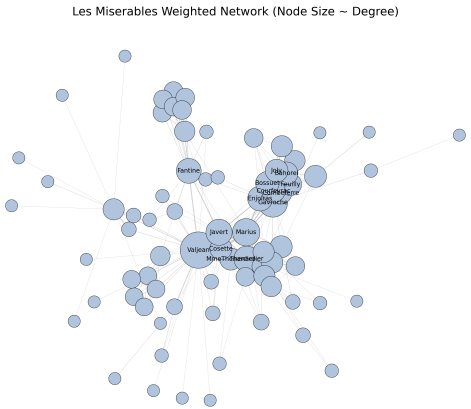
Part I Summary

- We introduced the dataset and drew a simple graph first.
- Then we examined node degree through a histogram and KDE.
- Finally, we compared algorithmic communities with the known real split.

Dataset 2

A weighted network with richer structure and more realistic complexity

Dataset Introduction



Basic Facts

Item	Value
Nodes	77
Edges	254
Average degree	6.60
Average strength	21.30
Graph type	Weighted
Ground truth	No

- Nodes are characters.
- Edge weights count co-occurrences.
- This is closer to a real application setting.

Part II Workflow

1. Introduce the weighted dataset.
2. Draw a weighted network visualization.
3. Compare degree and strength.
4. Perform community detection on the weighted graph.

Code: Load the Les Miserables Dataset

```
G2 = nx.les_miserables_graph()
lesmis_pos = nx.spring_layout(G2, seed=42, k=0.32)
lesmis_degree = dict(G2.degree())
lesmis_strength = dict(G2.degree(weight="weight"))

lesmis_degree_values = list(lesmis_degree.values())
lesmis_degree_min = min(lesmis_degree_values)
lesmis_degree_max = max(lesmis_degree_values)
```

Code: Build Node Sizes and Edge Widths

```
lesmis_node_sizes = {}
for node, degree_value in lesmis_degree.items():
    lesmis_node_sizes[node] = (
        250
        + (degree_value - lesmis_degree_min) * (2200 - 250)
        / (lesmis_degree_max - lesmis_degree_min)
    )

lesmis_edge_widths = []
for u, v in G2.edges():
    lesmis_edge_widths.append(0.2 + G2[u][v]["weight"] * 0.12)
```

Weighted Visualization

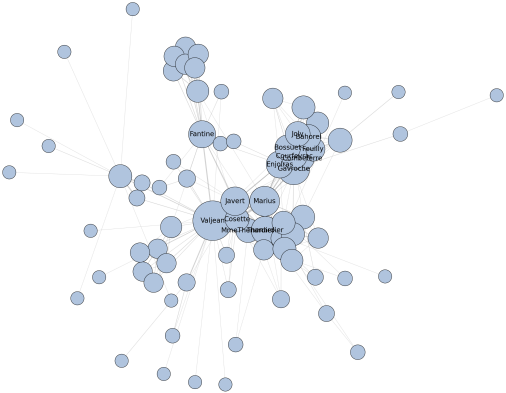
- Node size still represents degree.
- Edge width now represents weight.
- This adds one more layer of information compared with Karate Club.
- Dense weighted graphs are harder to read, so visual design matters even more.

Code: Draw the Weighted Network

```
nx.draw_networkx_edges(
    G2, lesmis_pos,
    alpha=0.35,
    edge_color="gray",
    width=lesmis_edge_widths
)
nx.draw_networkx_nodes(
    G2, lesmis_pos,
    node_size=[lesmis_node_sizes[node] for node in G2.nodes()],
    node_color="lightsteelblue"
)
```

Les Miserables: Weighted Network

Les Miserables Weighted Network (Node Size ~ Degree)



Degree vs Strength

- Degree counts how many neighbors a character has.
- Strength sums the weights of those relationships.
- In a weighted network, these two values are related but not identical.
- This helps explain why weights matter in real applications.

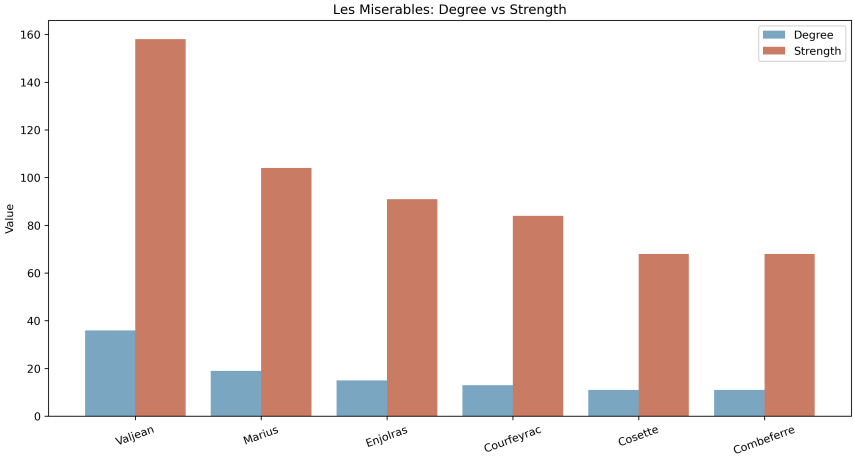
Code: Degree vs Strength Bar Chart

```
lesmis_top_strength_pairs = sorted(
    lesmis_strength.items(), key=lambda x: x[1], reverse=True
)[:6]
lesmis_top_names = [name for name, _ in lesmis_top_strength_pairs]
lesmis_top_degree_values = [lesmis_degree[name] for name in lesmis_top_names]
lesmis_top_strength_values = [lesmis_strength[name] for name in lesmis_top_names]
```

Code: Plot Degree and Strength

```
plt.bar([x - 0.2 for x in x_positions],
        lesmis_top_degree_values,
        width=0.4, label="Degree")
plt.bar([x + 0.2 for x in x_positions],
        lesmis_top_strength_values,
        width=0.4, label="Strength")
```

Les Miserables: Degree vs Strength



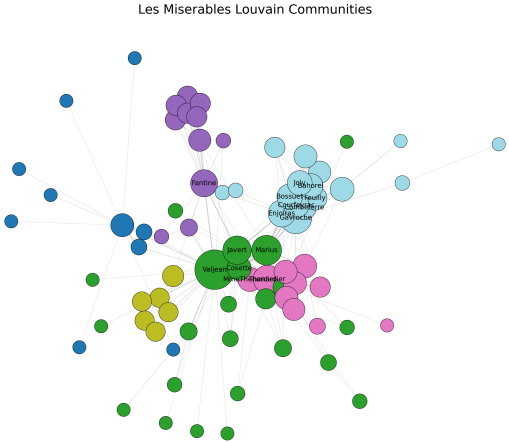
Weighted Community Detection

- Community detection is again the final analytical step.
- But now the algorithm should use edge weights.
- This allows stronger character relationships to influence the partition more.

Code: Louvain for the Weighted Graph

```
lesmis_louvain_partition = community_louvain.best_partition(
    G2, weight="weight", random_state=42
)
lesmis_louvain_modularity = community_louvain.modularity(
    lesmis_louvain_partition, G2, weight="weight"
)
lesmis_louvain_community_count = len(
    set(lesmis_louvain_partition.values())
)
```

Les Miserables: Community Detection Result



Part II Summary

- This dataset is more complex and more realistic than Karate Club.
- The weighted visualization shows not just who is connected, but how strongly.
- Degree and strength help us compare two forms of node importance.
- Weighted Louvain detection reveals character communities inside the larger story.

Final Comparison

Aspect	Karate Club	Les Miserables
Graph type	Unweighted	Weighted
Ground truth	Yes	No
Main analysis focus	Degree + label comparison	Degree, strength, weighted communities
Teaching style	Introductory and intuitive	Richer and more realistic

Conclusion

- The two datasets should be taught as two separate analysis stories.
- Karate Club works well for a clean introduction and for comparing detected communities with true labels.
- Les Miserables extends the discussion to weighted relationships and richer interpretation.
- Presenting code in the same order as the analysis makes the lecture easier to follow.

Goal: How does community detection work?

Problem:

- Given a network, group nodes into communities

Challenge:

- Graph structure is discrete and complex
- Hard to directly apply standard clustering

Idea:

$$\text{Graph} \Rightarrow \text{Vector Representation} \Rightarrow \text{Clustering}$$

Adjacency Matrix

We represent the graph by a matrix:

$$A \in \mathbb{R}^{n \times n}$$

- $A_{ij} = 1$ if edge exists
- $A_{ij} = 0$ otherwise

For weighted graphs:

- $A_{ij} =$ edge weight

Key Idea: Use Eigenvectors

Instead of working directly on the graph:

Compute eigen-decomposition:

$$A = U \Lambda U^\top$$

- Columns of U = eigenvectors
- Top eigenvectors capture main structure

Spectral Embedding

Take the top K eigenvectors:

$$U_K \in \mathbb{R}^{n \times K}$$

Embedding:

- Node $i \to$ row i of U_K

Result:

$$\text{Nodes} \to \text{points in } \mathbb{R}^K$$

Geometric Interpretation

After embedding:

- Similar nodes $\rightarrow$ close points
- Different communities $\rightarrow$ separated clusters

So we transformed:

$$\text{Graph problem} \Rightarrow \text{Clustering problem}$$

Clustering Step

Now apply standard clustering:

k-means:

- Input: rows of U_K
- Output: K clusters

Final output:

Community labels

Algorithm Summary

Spectral Clustering Pipeline:

1. Construct adjacency matrix A
2. Compute top K eigenvectors
3. Form embedding U_K
4. Apply k-means

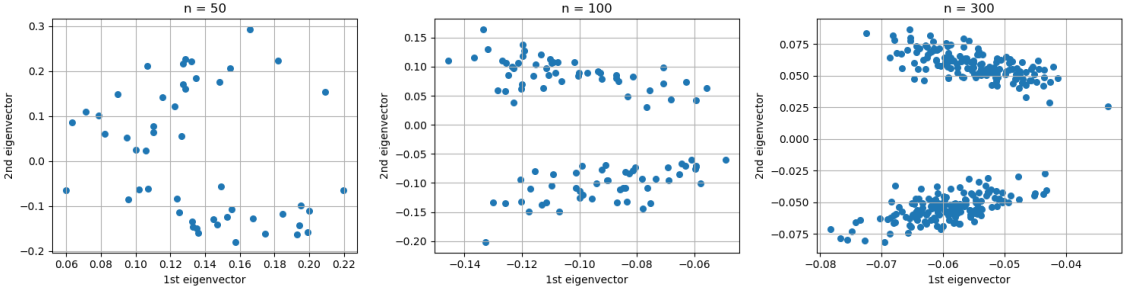
Simple and effective

Why Does It Work? (Intuition)

- Eigenvectors capture global structure
- They smooth out local noise
- Nodes with similar connectivity patterns become close

Key idea:

Connectivity patterns $\Rightarrow$ geometry



As number of nodes increases, two-dimensional embedding vectors get closer.